

Aula 04 - Atividade

Prática: Auditoria e Extensão do Painel de Logística

Modalidade: Individual ou em duplas

Formato de entrega: Arquivos modificados (`app.js` e `index.html`) + Folha de Defesa Técnica (txt ou pdf).

Parte 1: Implementação com Validação Personalizada (Mão no Código)

Você deve implementar duas novas melhorias no painel funcional construído em aula:

Requisito 1: Máscara de Rastreio Pessoal

- No envio do formulário, o sistema só deve aceitar códigos de rastreio que iniciem com as **suas iniciais** seguidas de hífen e número (exemplo para Lucas Silva: `LS-01`, `LS-02`).
- Se o código digitado não começar com suas iniciais em maiúsculo, o sistema deve disparar um `alert("Código inválido para este operador!")` e abortar o salvamento sem gravar nada no `localStorage`.

Requisito 2: Painel de Indicadores (Métricas no Topo)

- Logo acima da tabela (no HTML), adicione uma barra de contadores:
- Dentro da função `consultarEncomendas()` (ou logo após carregar os dados), calcule e atualize esses contadores no DOM automaticamente a cada inclusão, edição de status ou exclusão.

Parte 2: O Desafio do Bug Oculto (Análise de Código)

Um colega tentou criar um botão para "**Limpar Encomendas Entregues**", mas o código dele apresenta duas falhas conceituais e não funciona como esperado.

Analise o trecho abaixo:

JavaScript

```
// Tentativa do colega no app.js:
function limparEntregues() {
  const lista = localStorage.getItem("banco_encomendas");

  lista.forEach((item, index) => {
    if (item.status === "ENTREGUE") {
      lista.splice(index, 1);
    }
  });

  renderizarTabela(lista);
}
```

Responda na sua **Folha de Entrega**:

- Por que a linha `lista.forEach(...)` causará um erro fatal no console (`TypeError: lista.forEach is not a function`)? Qual função faltou executar na variável `lista`?
- Mesmo que a lista rodasse, por que as encomendas entregues voltariam a aparecer se o usuário der um F5 na página?

Parte 3: Evidência de Execução (DevTools)

Na sua **Folha de Entrega** anexe um print da tela inteira contendo:

- A tabela aberta com pelo menos 2 registros contendo o seu código personalizado (ex: `LS-01`).
- A aba **Application > Local Storage** aberta no DevTools (F12) mostrando a chave `banco_encomendas` e os mesmos dados salvos em formato de string JSON.

Parte 4: Defesa Técnica Relâmpago (3 Perguntas Rápidas)

Na sua **Folha de Entrega** responda:

1. **Ciclo de Conversão:** Para que serve exatamente o comando `JSON.stringify` na hora de salvar e o `JSON.parse` na hora de ler os dados do `localStorage`?
2. **Experiência do Usuário (UX):** Por que bloqueamos o botão com `btnSubmit.disabled = true` antes de esperar o salvamento acontecer? O que aconteceria se um usuário impaciente clicasse 5 vezes seguidas rápido?
3. **Arquitetura:** Se apagarmos a linha `await consultarEncomendas()` de dentro da função `cadastrarEncomenda()`, o dado é salvo? O que acontece com a tabela na tela imediatamente após o clique?

Rubrica de Avaliação

Critério	Excelente (100%)	Regular (50%)	Insuficiente (0%)
Validação Pessoal	Valida as iniciais do aluno antes de gravar no <code>localStorage</code> .	Cadastra qualquer código ou valida com erro de lógica.	Não implementou a validação.
Contadores no DOM	Métricas atualizam sozinhas em qualquer operação do CRUD.	Contadores só atualizam no F5 ou contam valores errados.	Não inseriu os contadores.
Análise do Bug	Identifica a ausência de <code>JSON.parse</code> e a falta de persistência no storage.	Identifica apenas 1 dos 2 problemas.	Não soube explicar o erro.
Defesa Técnica	Explica com clareza a serialização JSON e a sincronização do DOM.	Responde de forma vaga ou decoreba sem demonstrar domínio.	Não sabe explicar o próprio código.